Solution: Longest Consecutive Sequence

Problem Statement

Given an unsorted array of integers, find the length of the longest consecutive sequence of numbers in it. A consecutive sequence means the numbers in the sequence are contiguous without any gaps. For instance, 1, 2, 3, 4 is a consecutive sequence, but 1, 3, 4, 5 is not.

Examples

1.
    - **Input:** [10, 11, 14, 12, 13]
    - **Output:** 5
    - **Justification:** The entire array forms a consecutive sequence from 10 to 14.
2.
    - **Input:** [3, 6, 4, 100, 101, 102]
    - **Output:** 3
    - **Justification:** There are two consecutive sequences, [3, 4] and [100,101,102]. The latter has a maximum length of 3.
3.
    - **Input:** [4, 3, 6, 2, 5, 8, 4, 7, 0, 1]
    - **Output:** 9
    - **Justification:** The longest consecutive sequences here are [0, 1, 2,, 3, 4, 5, 6, 7, 8].
4.
    - **Input:** [7, 8, 10, 11, 15]
    - **Output:** 2
    - **Justification:** The longest consecutive sequences here are [7,8] and [10,11], both of length 2.

Solution

To solve this problem, the key observation is that if `n` is part of a consecutive sequence, then `n+1` and `n-1` must also be in that sequence.

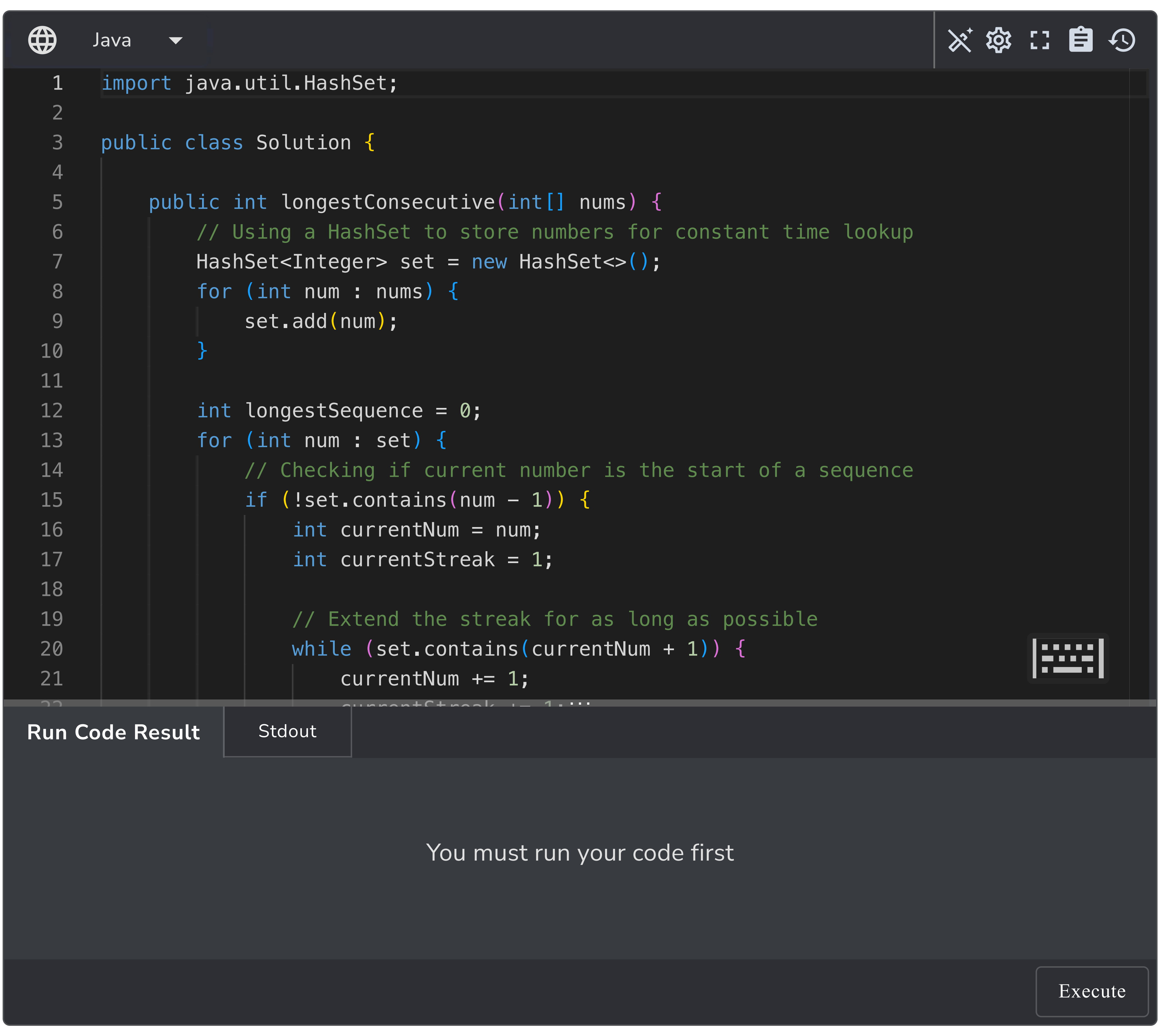
1. **HashSet**: Begin by inserting all elements of the array into a HashSet. The reason for using a HashSet is to ensure O(1) time complexity during look-up operations.
2. **Initial Scan**: Iterate through each element of the array. For every number, check if it's the starting point of a possible sequence. This can be determined by checking if `n-1` exists in the HashSet. If not, then it means `n` is the start of a sequence.
3. **Building Sequences**: For each starting number identified in step 2, keep checking if `n+1`, `n+2` ... exist in the HashSet. For each present number, increase the length of the sequence and move to the next number.
4. **Result**: Store the length of each sequence found in step 3. The answer will be the longest of all sequences identified.

Algorithm Walkthrough

Given the input: [3, 6, 4, 100, 101, 102]

- Initialize an empty HashSet and populate it with all numbers from the input.
- Start with the first number, 3. Since 2 (which is 3-1) is not in the HashSet, we recognize 3 as the starting of a sequence.
    - Check for 4. It's there. Move to 5. It's not there. So, the sequence is [3,4] with a length of 2.
- Next, take 6. 5 is not there, so 6 might be the start of a new sequence. However, 7 isn't in the HashSet. So, the sequence is just [6].
- For 100, 99 isn't there, so 100 is a starting point.
    - Check for 101. It's there. Check for 102. It's there. Check for 103. It's not there. The sequence is [100,101,102] with a length of 3.
- The algorithm returns 3, which is the length of the longest sequence.

Code



```java
import java.util.HashSet;

public class Solution {

    public int longestConsecutive(int[] nums) {
        // Using a HashSet to store numbers for constant time lookup
        HashSet<Integer> set = new HashSet<>();
        for (int num : nums) {
            set.add(num);
        }

        int longestSequence = 0;
        for (int num : set) {
            // Checking if current number is the start of a sequence
            if (!set.contains(num - 1)) {
                int currentNum = num;
                int currentStreak = 1;

                // Extend the streak for as long as possible
                while (set.contains(currentNum + 1)) {
                    currentNum += 1;
```

You must run your code first

Complexity Analysis

- **Time Complexity**: O(n). Although it seems that the while loop runs for each number, it only runs for the numbers that are the starting points of sequences. So, in total, each number is processed only once.
- **Space Complexity**: O(n). The space used by our set.

← Previous
Longest Consecutive Sequence (Medium)

Next →
Meeting Rooms II (Medium)

☐ Mark as Completed